Computational Linguistics 1 — IIIT Hyderabad

Code-Mixed Text Analysis

Hinglish & Tenglish NLP Pipelines — Full Report

Ram Kowshik & Aayush Swami | IIIT Hyderabad — April 2026

Git link-https://github.com/i-yush23/CL-1-project

Abstract

Code-mixing — the practice of alternating between two or more languages within a single sentence — is extremely common in Indian social media text. This report presents two rule-based and statistical NLP pipelines: (1) a Hinglish (Hindi-English) pipeline using Naive Bayes Language Identification (LID) and an HMM Part-of-Speech (POS) tagger, achieving 97.15% LID accuracy and 71.18% POS accuracy; and (2) a Tenglish (Telugu-English) pipeline adapted from the Hinglish system, achieving 65.78% POS accuracy and 55.33% LID accuracy on 5,892 evaluation tokens. Both pipelines share a common phonetic-hashing architecture and Viterbi decoding strategy. The Hinglish pipeline additionally incorporates three statistical modules — a CMI Calculator, Emoji Analyzer, and Matrix Language Classifier — which extend purely token-level analysis to sentence-level linguistic characterisation. The comparison reveals that the HMM architecture transfers well across language pairs, but Telugu-specific romanisation variability makes Tenglish LID substantially harder.

1 What is Code-Mixing

Think about how people actually talk in a city like Hyderabad or Mumbai. Someone will start a sentence in Telugu or Hindi and finish it in English without even thinking about it. That is code-mixing. It is not a mistake or a bad habit — it is just how multilingual people naturally speak and write.

More formally, code-mixing happens when a speaker uses words or phrases from two or more languages in the same sentence. Sridhar and Sridhar (1980) describe it as moving from the words of one language to the words of another within a single sentence. It differs slightly from code-switching, where a person changes languages between sentences rather than inside one; in practice both terms are used interchangeably (Wikipedia, 2024).

In India this is not a niche phenomenon. India has 22 official languages listed in the Eighth Schedule of the Constitution and over 121 mother tongues recorded in the 2011 census. English has become a common working language alongside Hindi, Telugu, Tamil, Kannada, Bengali, and many others. Varieties like Hinglish (Hindi-English) and Tenglish (Telugu-English) appear everywhere on social media, YouTube comments, and WhatsApp chats. A message like "yaar I'm so thak gaya from this semester" is completely normal to a multilingual speaker, but for most NLP software it is essentially unreadable.

2 Relevance to Computational Linguistics

NLP tools have improved dramatically in recent years, but almost all of that progress relied on clean, formal, single-language text. When standard NLP tools encounter code-mixed text they struggle badly: an English tokeniser has no idea what to do with a Hindi word written in Roman script; a sentiment model trained on English tweets misses emotion expressed in another language; even basic language detection is hard because the same Roman script is being used for multiple languages at once.

Srivastava and Singh (2021) document how these failures appear across crisis management, healthcare, fake news detection, and more.

The first step toward fixing any of this is normalisation — cleaning and standardising raw text so that downstream tools can process it. For code-mixed text this means dealing with non-standard spelling, word-by-word language boundary detection, emoji, and slang. These are exactly the stages our pipelines are designed to handle.

3 System Architecture

Both pipelines share a common five-stage processing architecture. Language-specific differences are noted where relevant.

3.1 Tokenisation

Input text is split using a regular expression that preserves contractions and separates punctuation as individual tokens. The Hinglish pipeline additionally routes emoji characters to a sentiment annotation module; the Tenglish pipeline uses the same regex without that layer.

```
tokens = re.findall(r"[\w']+|[.,!?;:—…]", text)
```

3.2 Phonetic Normalisation

The phonetic hasher converts each token to a canonical skeleton, dramatically reducing the effective vocabulary size. It operates in four steps:

- Repeated-character compression: aa→a, ee→e, oo→o, dd→d, tt→t
- Multi-character cluster substitution: sh→S, ph→F, ch→C, bh→B, kh→K (applied before single-char rules)
- Single-char phoneme grouping: k/c/q→K, v/w→W, s/z→S
- Vowel dropping: first character kept; all vowels removed from the remainder

The Hinglish hasher collapses 66,828 surface forms into 22,803 equivalence classes (33% compression). The Tenglish hasher applies the same rules with additional Telugu-specific handling for retroflex consonants and long vowels.

3.3 Language Identification

Each token is classified as EN, HI/TE, or UNI through a cascade. The Hinglish pipeline uses a Naive Bayes model; the Tenglish pipeline uses a rule-based segmenter. Both share the same cascade logic:

Step	Rule	Hinglish	Tenglish
1	Hard rule: punct, digits, URLs, hashtags, @mentions	→ UNI	→ UNI
2	Transliteration fallback dict	transliteration_variants.json	transliteration_variants.json
3	Indicator set membership	EN / HI word lists	EN / TE word lists
4	Phonetic hash fallback	22,803 buckets	Pre-built hash sets
5	Context smoothing (+/−2 window)	softmax blend (weight=0.15)	neighbour-majority vote
6	Default for unknowns	→ HI (class prior)	→ TE (Telugu-dominant OOV)

3.4 POS Tagging — HMM Viterbi

A first-order Hidden Markov Model is trained on annotated token data in both pipelines. Emission probabilities P(word | tag) use a three-tier lookup: exact word match, phonetic hash fallback, and tag-frequency prior. Transition probabilities P(tag_t | tag_{t-1}) use bigram counts with a START state. Viterbi decoding finds the most probable sequence; a floor of 1e-10 prevents zero-probability paths. Hard overrides ensure punctuation tokens always receive PUNC regardless of Viterbi output.

3.5 Emoji Annotation (Hinglish only)

The Hinglish pipeline includes an Emoji Analyser that maps emoji to POSITIVE, NEGATIVE, or NEUTRAL placeholders using a 1,591-entry custom lexicon. This stage is absent from the Tenglish pipeline.

4 Datasets

4.1 Hinglish

Training data originates from a Hinglish CoNLL/BIS-annotated corpus. The LID corpus was split 80/20 (seed 42). The class distribution is Hindi-dominant: HI ≈71.3%, EN ≈27.5%, UNI ≈1.5%.

Split	Tokens	Sentences
LID Test	271,551	8,891
POS Train	16,796	—
POS Test	4,032	312

4.2 Tenglish

The dataset was sourced from the ICON 2016 Shared Task on POS Tagging for Code-Mixed Indian Social Media Text (Telugu-English Facebook comments). The original 3-column space-separated format (token, POS_tag, LID_label) was converted to CSV with universal tagset mapping.

Split	Tokens	Sentences
Training	23,560	1,584
Test	5,892	392
Total	29,452	1,976

Training set LID distribution (Tenglish):

Language	Count	Percentage
English (EN)	11,246	47.7%
Telugu (TE)	6,493	27.6%
Universal (UNI)	5,821	24.7%

5 Challenges in Code-Mixed NLP

5.1 Shared Challenges

No standard romanisation. The same word appears in multiple spellings (Hindi: bhai/bhaai/bhya; Telugu: vaadu/vadu/vaaduu/wadu). Addressed by phonetic hashing.

Ambiguous short words. Short tokens appear in both languages with different meanings (e.g. hi = English greeting / Hindi emphatic particle; lo = Telugu postposition / English slang). Addressed by context smoothing.

Proper nouns. Named entities like Hyderabad, Prabhas, Garuda are neither purely one language nor the other; the LID system often misclassifies them. Future fix: named-entity detection.

Social media noise. Emoticons, hashtags, URLs, repeated punctuation, and mixed-script characters. Addressed by the _is_universal() hard rule, which captures non-alpha tokens first.

Class imbalance. Noun (N) dominates both training sets, biasing the HMM and reducing recall for minority classes.

5.2 Tenglish-Specific Challenges

No official Telugu romanisation. Retroflex consonants and long vowels create a larger spelling-variant space than Hindi, e.g. cheppaadu/cheppadu/chepadu all map to the same CHPD hash.

Postpositions fused to English words. *officelo* (office+lo = "in the office"), *meetingki* (meeting+ki = "to the meeting"). Fused tokens look like Telugu words to the LID system. Future fix: morphological segmentation.

Smaller pre-built resources. The Telugu indicator list required manual construction from scratch (161 entries), vs. the more mature Hinglish resources.

5.3 Hinglish-Specific Challenges

English function word absorption. Short English words (to, the, a, is) appear in Hindi-dominant contexts in training, so the Hindi-biased class prior pulls them toward HI, producing 88.8% of all LID errors.

Tagset mismatch. Rare tags ($, E, null) have zero support in the test set and score 0% F1, slightly suppressing reported overall POS accuracy.

6 Hinglish Pipeline — Results

The Hinglish pipeline was evaluated on (a) the LID test set of 271,551 tokens across 8,891 sentences, and (b) the BIS/IIIT POS test set of 4,032 tokens across 312 sentences.

6.1 Language Identification

Overall LID Accuracy: 97.15% (263,809 / 271,551 tokens correct)

Language	Precision	Recall	F1-Score	Support
EN	98.72%	90.73%	94.56%	74,135
HI	96.55%	99.55%	98.03%	193,475

UNI	100.00%	100.00%	100.00%	3,941

UNI achieves perfect 100% because non-alphabetic tokens are deterministically classified. HI achieves near-perfect recall (99.55%) at a slight precision cost due to the Hindi-biased class prior. EN has strong precision (98.72%) but lower recall (90.73%): 9.3% of English tokens are pulled toward HI. Of the 7,742 total errors, EN→HI accounts for 6,872 (88.8%) and HI→EN for 870 (11.2%).

6.2 POS Tagging

Overall: 71.18% ■ **In-Vocabulary: 79.62%** ■ **OOV: 31.00%** (4,032 tokens; 3,332 in-vocab, 700 OOV)

Tag	Full Name	Prec.	Rec.	F1	Support
N	Noun	71.5%	75.2%	73.3%	819
V	Verb	73.0%	80.4%	76.5%	774
PSP	Postposition	70.6%	76.0%	73.2%	421
PRP	Pronoun	71.2%	87.0%	78.3%	400
DT	Determiner	85.5%	91.0%	88.1%	278
J	Adjective	74.0%	51.7%	60.9%	259
G_N	Genitival Noun	63.3%	47.1%	54.0%	238
R	Adverb	73.8%	77.2%	75.5%	233
CC	Conjunction	69.2%	52.3%	59.6%	155
PRT	Particle	57.3%	60.8%	59.0%	148
G_V	Genitival Verb	59.8%	51.9%	55.6%	106
X	Other	45.0%	17.3%	25.0%	52

DT achieves the highest F1 (88.1%) due to a small closed set of determiners. The N↔V confusion (103+74=177 errors) and J→N confusion (61 errors) reflect genuine structural ambiguity: many words function as both nouns and verbs in romanised text (e.g. kaam = work / to-work). The 48.6-point gap between in-vocabulary and OOV accuracy (79.62% vs. 31.00%) highlights how critical spelling normalisation is.

6.3 Top Confusion Pairs

True	Predicted	Count	Interpretation
N	V	103	Noun/verb base-form overlap (kaam = work/to-work)
V	N	74	Mirror of above
J	N	61	Adjectives before nouns re-labelled N
CC	PSP	42	Conjunctions vs. postpositions both follow NPs
G_N	N	39	Genitive noun collapses to plain N for OOV
PSP	CC	28	Overlap on words like aur, se
V	PRP	28	Short verbs ho/kar used pronominally

7 Hinglish Pipeline — Additional Statistical Modules

The Hinglish pipeline was extended with three analytical modules that go beyond token-level LID and POS tagging to provide sentence-level linguistic characterisation. These modules operate on the outputs of the core pipeline stages.

7.1 Code-Mixing Index (CMI) Calculator

File: `metrics/cmi_calculator.py`

The CMI module quantifies the level of language mixing within a given sentence. It is a standard metric in multilingual NLP used to determine how heavily a sentence relies on code-switching rather than being predominantly monolingual.

How it works. The text is tokenised and passed through the CodeswitchSegmenter, which labels every token as HI, EN, or UNI (universal, e.g. punctuation or numbers). It then tallies occurrences of each label and computes CMI using:

$$CMI = 100 \times (1 - \max(wHI, wEN) / (N - U))$$

where N is the total token count, U is the UNI token count, and wHI / wEN are the per-language token counts.

Output. The module returns a structured dictionary containing: the final CMI score (0.0–100.0, where 0 means fully monolingual), total token count, per-language token counts (HI / EN / UNI), and token-label pairings for transparency.

7.2 Emoji Analyzer

File: `features/emoji_analyzer.py`

Social media data such as Hinglish tweets and chats is heavily populated with emojis. Statistical models like HMMs or CRFs often struggle with raw unicode characters. The Emoji Analyzer detects these symbols and translates them into a statistical-model-friendly format.

Key features: (a) Robust Unicode matching using a comprehensive regex covering emoticons, transport/map symbols, dingbats, and supplemental pictographs. (b) Modifier handling — Variation Selectors (\ufe0f, \ufe0e) and Zero-Width Joiners (\u200d) are stripped during processing. (c) Sentiment mapping using a predefined emoji_lexicon.json that maps specific emojis to POSITIVE, NEGATIVE, or NEUTRAL, converting each emoji to a standardised placeholder token such as __EMOJI_POSITIVE__.

Pipeline integration. The module executes early in the pipeline. By converting unpredictable unicode emojis into predictable placeholder tokens, the downstream POS tagger and LID segmenter can process sentence structure more reliably without crashing or producing out-of-vocabulary errors.

7.3 Matrix Language Classifier

File: `statistical_models/matrix_classifier.py`

Based on the Matrix Language Frame (MLF) model in linguistics, this module determines which language provides the grammatical backbone (the Matrix Language) and which language merely contributes vocabulary (the Embedded Language) in a code-switched sentence.

How it works. Grammatical function words — verbs, auxiliary verbs, conjunctions, determiners, pronouns, and prepositions/postpositions — are the strongest indicators of the matrix language. The module uses predefined POS tag sets (HI_FUNCTION_POS and EN_FUNCTION_POS) to identify these anchor tokens, then applies structural weighting: function words receive weight 2.0, content words receive weight 1.0. It calculates a weighted hi_score and en_score. If the scores are within a 15% threshold of each other, the language is classified as BALANCED; otherwise the language with the higher structural score is declared the Matrix Language.

Output. The module returns a structured dictionary containing: matrix_language ("HI", "EN", or "BALANCED"), embedded_language (the opposing language, or "NONE" if purely monolingual), hi_score and en_score, and an evidence list of the specific anchor tokens used — including their LID label, POS tag, and applied weight.

8 Tenglish Pipeline — Results

The Tenglish pipeline was evaluated on 5,892 tokens from 392 sentences of Telugu-English Facebook comments and Twitter posts (ICON 2016 Shared Task). Results below reflect the final project submission; accuracy improvements relative to the mid-project submission are documented in Section 8.4.

8.1 Accuracy Summary

Task	Mid Project	Final Project	Change
POS Tagging	49.8%	63.5%	+13.7 pp
Language ID (LID)	55.1%	60.4%	+5.4 pp
POS Tag Count	22 tags	12 tags	−10 tags
Known Vocabulary	4,885 words	6,753 words	+1,868 words

8.2 POS Tagging

Overall POS Accuracy: 63.5% (final project, 5,892 tokens)

Tag	Description	Mid F1	Final F1	Change	Support
ADJ	Adjective	0.176	0.686	+0.510	345
ADP	Adposition	0.568	0.454	−0.114	234
ADV	Adverb	0.400	0.585	+0.185	131
CONJ	Conjunction	0.593	0.624	+0.031	51
DET	Determiner	0.517	0.631	+0.114	132
INTJ	Interjection	0.000	0.480	+0.480	34
N	Noun	0.514	0.707	+0.193	2,179
PART	Particle	0.009	0.249	+0.240	144
PRON	Pronoun	0.414	0.489	+0.075	191
PUNC	Punctuation	0.625	0.821	+0.196	767
V	Verb	0.463	0.610	+0.147	562
X	Foreign/unclassified	0.456	0.532	+0.076	1,122

N (F1=0.707) and PUNC (F1=0.821) show the strongest final scores. ADJ improved most dramatically (+0.510), driven by tagset consolidation. The ADP dip (−0.114) is expected: tokens previously mis-tagged as ADP are now correctly intercepted as PUNC or X. INTJ recovered from 0.000 (phantom tag in mid-project) to 0.480 in the final submission.

8.3 Language Identification

Overall LID Accuracy: 60.4% (final project, 5,892 tokens)

Language	Mid F1	Final F1	Change	Key Reason
EN	0.533	0.607	+0.074	Normalisation fallback lookup
TE	0.574	0.718	+0.144	Unicode rule + geminate fix
UNI	0.539	0.506	−0.033	Tokens promoted to TE/EN

TE F1 improved the most (+0.144), driven by the Telugu Unicode hard rule and geminate-preservation fix. EN F1 improved via the normalisation fallback. The UNI dip (−0.033) is intentional and correct: the model now confidently assigns more tokens to TE or EN rather than falling back to UNI, reducing false UNI labels.

8.4 Code Changes (Mid → Final)

All accuracy gains are attributable exclusively to logic changes in two files: hmm_pos_tagger.py and code_switch_labeler.py. No architectural changes, retraining, or new data were involved. The improvements fall into four categories.

Punctuation & noise handling (hmm_pos_tagger.py). The hard-coded punctuation set was extended to cover social-media tokens (.., ????, :), xD, \m/, > etc.), eliminating spurious Viterbi assignments for these tokens. Two regex rules were added: URLs matching *.com/org/net/* are returned as X; symbol-only sequences of length ≥2 are returned as PUNC. Tokens like pic.twitter.com/j5loiORCjM — previously classified as NUM — are now correctly labelled X. PUNC F1 improved from 0.625 → 0.821.

Lexicon completeness (hmm_pos_tagger.py). The Telugu pronoun hint list was expanded with high-frequency interrogative and personal pronouns: idi, evaru, emiti, emi, ela, ekkada, eppudu, mee, naa, ninnu, nannu. English conjunctions either, neither, while, since and Telugu conjunctions mariyu, ledha, inkaa, kaabatti, alage, anduvalla were added. Telugu postpositions dwara, valla, tarvata, mundu, lopala, bayata, daggara were added. These are closed-class words; a complete lexicon eliminates almost all errors for their categories. PRON F1 improved from 0.414 → 0.489; CONJ F1 from 0.593 → 0.624.

Normalisation correctness (code_switch_labeler.py). The elongation collapse threshold was raised from 3+ repeated characters to 4+, preserving Telugu geminates: nuvvu, cheyyi, vachchi survive preprocessing and hit the indicator lookup correctly. A second normalised-form fallback lookup was added after the primary indicator check, catching forms like heyyyy where the once-collapsed heyy is not in the EN list but the twice-normalised hey is. EN F1 improved from 0.533 → 0.607. A Telugu Unicode hard rule (_TELUGU_RE = re.compile(r'[\u0C00-\u0C7F]')) was added: any token containing a

character in the Telugu Unicode block immediately returns TE before any other logic runs, giving zero-error classification for native-script tokens.

Context resolution quality (code_switch_labeler.py). The context-smoothing window was expanded from ±2 to ±3 neighbours. The simple majority vote was replaced by a weighted vote: offset-1 neighbours carry weight 2, offset-2 carry weight 1, offset-3 carry weight 0. A sentence-level majority fallback handles tokens with no informative neighbours (e.g. proper nouns at sentence boundaries) rather than defaulting blindly to TE. Mixed alphanumeric tokens (100rs, 2pm, no1) are now flagged as UNI before the LID cascade, preventing spurious votes from polluting the smoothing window. TE F1 improved to 0.718.

9 Comparative Analysis — Hinglish vs. Tenglish

Having both pipelines on the same base architecture allows a direct comparison. The results confirm that the HMM framework transfers effectively while clearly showing where Telugu-specific characteristics make the problem harder.

Dimension	Hinglish	Tenglish
Training tokens	16,796 (POS) / ~217k (LID)	23,560
Test tokens	4,032 (POS) / 271,551 (LID)	5,892
POS accuracy	71.18%	63.5% (final) / 39.8% (mid)
LID accuracy	97.15%	60.4% (final) / 55.1% (mid)
LID method	Naive Bayes	Rule-based cascade
Indicator vocabulary	66,828 surface forms	161 TE + 235 EN
Phonetic hash buckets	22,803	Pre-built hash sets
Tagset	BIS/IIIT (21 tags)	Universal (14 tags)
Best POS tag	DT — F1 0.881	INTJ — F1 0.892
Worst POS tag	X — F1 0.250	X — F1 0.558
Dominant LID error	EN→HI (88.8% of errors)	EN→TE (very high)
Main LID bottleneck	Class prior Hindi bias	Small indicator list
Main POS bottleneck	OOV (31% OOV accuracy)	N-dominance + spelling variants
Hardest feature	Hindi script variants	Telugu romanisation, no standard spelling
Additional modules	CMI, Emoji Analyzer, Matrix Language Classifier	—

Tenglish is inherently harder than Hinglish for LID for three reasons: (i) Hindi has a larger NLP community and more pre-built resources; (ii) Telugu romanisation is more variable due to retroflex consonants and long vowels; (iii) the Telugu indicator list required manual construction from scratch. Despite this, the Tenglish POS tagger in its final form (63.5%) is only 7.7 percentage points behind Hinglish, and targeted code-only fixes — no retraining, no new data — raised it by 23.7 pp over the mid-project baseline.

10 Key Findings

F1 — Near-flawless Hinglish LID for a purely statistical model. Achieving 97.15% token-level LID accuracy without pre-trained embeddings or neural networks is a strong result. Naive Bayes with

Laplace smoothing, phonetic hashing, and a small context window prove highly effective for the EN/HI binary decision in romanised text.

F2 — Phonetic generalisation is the critical shared innovation. The 66,828→22,803 vocabulary collapse (66% reduction) is the model's most impactful design decision. Without phonetic folding, OOV tokens — which make up a substantial fraction of code-mixed text — would be classified by class priors alone. The same principle applies in Tenglish.

F3 — Class prior bias drives systematic EN→majority-language errors. 88.8% of Hinglish LID errors are EN→HI. In Tenglish, EN→TE is similarly dominant. Both trace to the same root cause: the majority-language prior overwhelms weak evidence for English function words. Cross-lingual word lists or character n-grams could mitigate this.

F4 — HMM POS performs well on known vocabulary. In-vocabulary Hinglish POS accuracy of 79.62% is competitive for a statistical model without morphological features. The Viterbi decoder learns dominant patterns well across both tagsets.

F5 — OOV is the primary POS bottleneck. Hinglish OOV accuracy drops to 31.00%, a 48.6-point gap vs. in-vocabulary. The tag-frequency prior fallback provides a reasonable N/V/PSP bias for unknown words, but cannot capture morphological cues. This is compounded in Tenglish by the lack of a standard spelling system.

F6 — Structural ambiguity drives most POS errors. The N↔V confusion (177 errors in Hinglish) and N-dominance in Tenglish both reflect that many words function as both nouns and verbs in romanised text, and adjective-noun agreement is less morphologically marked than in native script.

F7 — Deterministic tokens are perfect. UNI (punctuation, numbers, symbols) achieves 100% F1 in Hinglish. The non-alpha → UNI hard rule is correct by construction and provides a zero-error floor in both pipelines.

F8 — Sentence-level modules extend the analytical scope of the Hinglish pipeline. The CMI Calculator, Emoji Analyzer, and Matrix Language Classifier provide richer linguistic characterisation beyond per-token labels. The CMI score quantifies mixing intensity; the Emoji Analyzer prevents OOV failures on unicode characters; the Matrix Language Classifier identifies the grammatical backbone language using structural weighting, enabling downstream applications such as grammar checking and style analysis.

11 Limitations

Scope	Limitation	Detail
Both	No character n-gram features	LID could benefit from subword character trigrams for better transliteration discrimination.
Both	First-order Markov	The HMM only considers the preceding tag; a second-order model would capture longer-range dependencies.
Both	Static transliteration dict	The variants JSON covers only a small fraction of common variants; a learned edit-distance model would scale better.
Both	No sentence-level priors	Token-by-token operation misses sentence-level language dominance signals.

Hinglish	POS tagset mismatch	Rare tags ($, E, null) have zero test support and score 0% F1, slightly suppressing overall metrics.
Tenglish	Small indicator lists	Only 161 TE + 235 EN entries; many common English words default to TE, severely limiting LID recall.
Tenglish	No morphological segmentation	Fused tokens like officelo are treated as single tokens, causing systematic errors for this token class.

12 Conclusion

This report presented two complete NLP pipelines for Language Identification and POS Tagging of code-mixed Indian social media text: a Hinglish system achieving 97.15% LID accuracy and 71.18% POS accuracy, and a Tenglish system achieving 63.5% POS accuracy and 60.4% LID accuracy in its final form — both built without neural networks or pre-trained embeddings.

The primary finding across both systems is that the combination of phonetic hashing, statistical modelling, and sliding-window context smoothing is surprisingly effective for romanised code-mixed text. The phonetic hashing step is the single highest-impact component, collapsing spelling variants into equivalence classes and dramatically reducing effective OOV rates.

The Hinglish pipeline is further extended with three statistical modules — CMI Calculator, Emoji Analyzer, and Matrix Language Classifier — that provide sentence-level linguistic characterisation. These modules transform the pipeline from a pure annotation tool into a richer analytical system capable of quantifying code-mixing intensity, handling social media emoji robustly, and identifying the grammatical backbone language in each utterance.

The comparison also reveals a clear resource gap. Hinglish benefits from a large vocabulary (66,828 surface forms) and mature training data; Tenglish started with only 161+235 indicator words. The LID accuracy gap (97.15% vs. 60.4% final) reflects this substantially. The POS accuracy gap (71.18% vs. 63.5% final) is much smaller, confirming that the HMM architecture is sound and that targeted logic fixes alone — without retraining or new data — can recover significant accuracy.

Future Work

Scope	Direction	Description
Both	Expand indicator lists	Add 500+ words to both te_indicators.json and en_indicators.json.
Both	Named entity detection	Pre-label proper nouns as UNI before LID runs.
Both	CRF-based LID	Replace rule-based segmenter with a CRF (F1 ~0.91 on similar data).
Both	Transformer-based POS	Fine-tune MuRIL or IndicBERT on code-mixed data.
Hinglish	Character n-gram LID	Add subword character trigrams to the Naive Bayes feature set.
Hinglish	Second-order HMM / MEMM	Capture longer-range tag dependencies.
Tenglish	Morphological segmentation	Split fused tokens (officelo → office + lo) before tagging.
Tenglish	Auto indicator expansion	Mine training corpus for missing indicators via phonetic clustering.

References

1. Association for Asian Studies. (2023). Multilingualism in India. Education About Asia.
2. Das, A., & Gambäck, B. (2014). Identifying languages at the word level in code-mixed Indian social media text. Proceedings of ICON 2014 (pp. 378–387).
3. Das, A. et al. (2016). ICON 2016 Shared Task: POS Tagging for Code-Mixed Indian Social Media Text. ICON 2016, IIT (BHU) Varanasi.
4. Gundapu, S. (2018). Word-Level Language Identification in English-Telugu Code-Mixed Data. GitHub Repository.
5. Khurana, D., Koli, A., Khatter, K., & Singh, S. (2022). Natural language processing: State of the art, current trends and challenges. PMC – National Library of Medicine.
6. Kunchukuttan, A. et al. (2020). AI4Bharat IndicNLP Catalog. GitHub.
7. Myers-Scotton, C. (1997). Code-switching. In F. Coulmas (Ed.), The Handbook of Sociolinguistics. Blackwell.
8. Poplack, S. (1980). Sometimes I'll start a sentence in Spanish y termino en Español. Linguistics, 18(7–8), 581–618.
9. Solorio, T. & Liu, Y. (2008). Learning to Predict Code-Switching Points. EMNLP 2008.
10. Sridhar, S. N., & Sridhar, K. K. (1980). The syntax and psycholinguistics of bilingual code mixing. Canadian Journal of Psychology, 34(4), 407–416.
11. Srivastava, V., & Singh, M. (2021). Challenges and considerations with code-mixed NLP for multilingual societies. arXiv:2106.07823.
12. Vyas, Y. et al. (2014). POS Tagging of English-Hindi Code-Mixed Social Media Content. EMNLP 2014.
13. Wikipedia. (2024). Code-mixing. https://en.wikipedia.org/wiki/Code-mixing
14. Wikipedia. (2026). Languages of India. https://en.wikipedia.org/wiki/Languages_of_India

Word count (combined): ~5,400 words ■ Pipelines tested on: Hinglish romanised corpus & Telugu-English Facebook/Twitter comments (ICON 2016)